



FIREBRAND

LYCEUM EDUCATION GROUP

Data Structures and Their Application

Python Pathway | Session 4

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

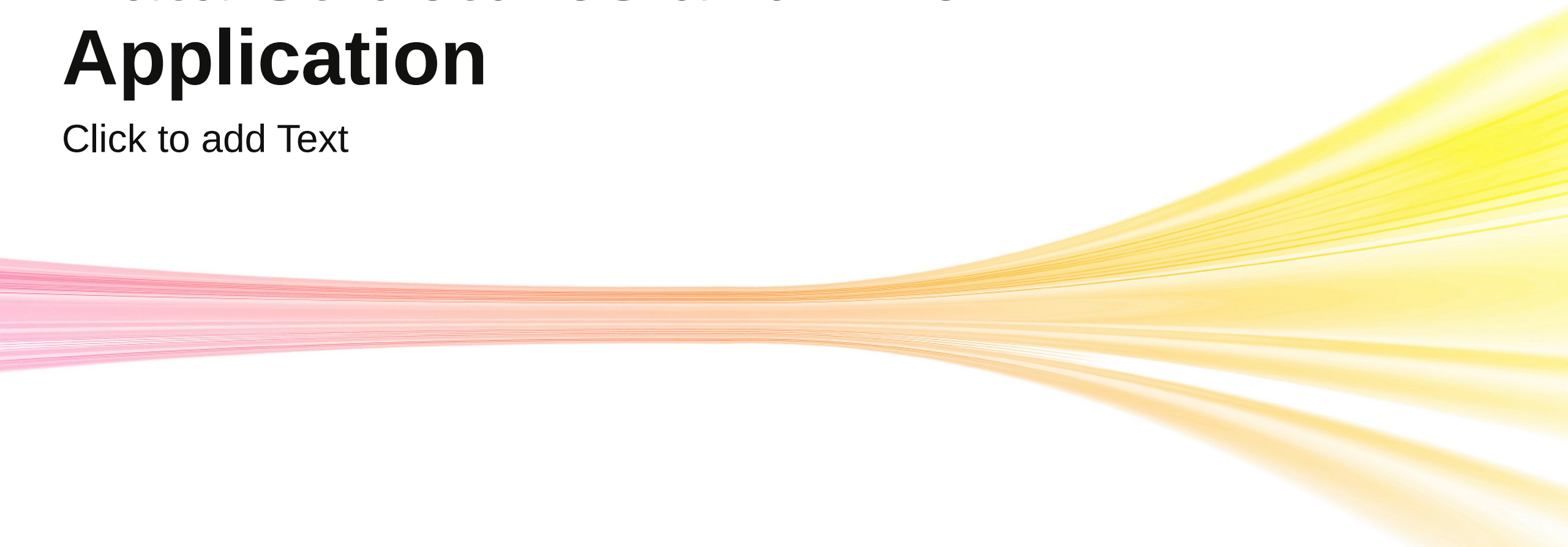
- Analyse differences between lists, dictionaries, and tuples

Evaluate

- Evaluate structure suitability based on problem context

Data Structures and Their Application

Click to add Text

A decorative graphic consisting of several overlapping, wavy lines that originate from the left side of the slide and extend towards the right. The lines are colored in a gradient of pink, orange, and yellow, creating a sense of motion and energy.

Introduction to Data Structures

Core Concepts

- **Lists** hold sequential items:

List: Ordered and mutable collection

```
shopping_list = ["Apple", "Banana", "Cherry"]
```

- **Tuples** protect permanent groupings:

Tuple: Fixed, immutable sequence

```
coordinates = (54.97, -1.61)
```

- **Dictionaries** power lightening-fast lookups:

Dictionary: Key-value map pairs

```
user_profile = {"id": 101, "role": "Admin"}
```

Structural Differences

Feature	Lists ([])	Tuples (())	Dictionaries ({ })
Ordering	Ordered sequence	Ordered sequence	Ordered by insertion
Mutability	Mutable (Changeable)	Immutable (Locked)	Mutable keys/values
Access Mode	Integer index ([0])	Integer index ([0])	Unique Key (["id"])
Duplicates	Allowed entries	Allowed entries	Duplicate keys banned

Syntax Traps

- Lists use **square** brackets
[]
- Tuples use **round** parentheses
()
- Dictionaries use **curly** braces
{ }

Strengths and Constraints

❑ **Lists: Dynamic Growth**

- **Strength:** Easily append elements
- **Constraint:** Slower lookups in large datasets

❑ **Tuples: Read-Only Safety**

- **Strength:** Write-protected memory blocks
- **Constraint:** Cannot alter sizes after initialisation

❑ **Dictionaries: Key-Value Speed**

- **Strength:** Instant data element lookups
- **Constraint:** Uses significant system memory overhead

Mutability
vs.
Rigid Protection

Real-World Use Cases

☐ **Lists: Ordered Data Logs**

- Track incoming sensor streaming data
- Manage linear queue processing lists

☐ **Tuples: Grouped Record Constants**

- Store permanent latitude/longitude coordinate data
- Define fixed RGB colour palette values

☐ **Dictionaries: High-Speed Lookups**

- Build server status lookup systems
- Manage configuration settings map tables

Mapping Code
to
Business Needs

Right vs. Wrong Collections

Inappropriate Choices:

- **Bad:** Using a list to look up prices by item name (requires checking every item sequentially)
- **Bad:** Using a tuple for user shopping carts (crashes when adding items)

Appropriate Replacements

- **Good:** Use dictionaries for rapid keyword lookups
- **Good:** Use lists for fluid cart additions



FIREBRAND

LYCEUM EDUCATION GROUP

Hands-On Exploration

Practical Exercises: Manipulating Collections

Core Tasks:

- **Modify lists:** Add, remove, and alter elements inside arrays
- **Update dictionaries:** Insert new key-value pairs and change records
- **Access sequences:** Extract elements using numeric indexes and unique keys
- **Loop collections:** Traverse all data structures using iterative structures

Expected Outcome:

- **Working scripts:** Programs execute smoothly without throwing exceptions
- **Correct types:** Code utilises the correct data structure for each task

Task 1: List Operations

```
# Create an inventory log list
inventory = ["Laptop", "Monitor", "Keyboard"]

# TODO: Add "Mouse" to the end of the inventory list
# TODO: Remove "Monitor" from the collection
# TODO: Update the first item in the list to "Premium Laptop"

print(f"Updated Inventory: {inventory}")
```

Mechanics to Observe:

- **Zero-indexing:** Accessing the first item requires index 0
- **Dynamic resizing:** The list length shifts automatically after modifications

Task 2: Dictionary Lookups

```
# Define a server profile database entry
server = {"host": "192.168.1.1", "status": "Online"}

# TODO: Add a new key "port" with the value 8080
# TODO: Update the "status" key to "Maintenance"
# TODO: Access and print the "host" value using its
key

print(f"Updated Server Data: {server}")
```

Mechanics to Observe:

- **Key collisions:** Assigning a value to an existing key overrides the old data
- **Banned indexing:** Attempting `server[0]` will fail unless 0 is an explicit key

Task 3: Iteration Experiments

```
# List of server location coordinates (Tuples)
locations = [(54.97, -1.61), (51.50, -0.12), (53.48,
-2.24)]

# TODO: Use a for loop to iterate over the locations
list

# TODO: Inside the loop, extract and print the
individual coordinates
```

The Tuple Constraint Challenge:

- **Experiment:** Try to alter a tuple inside your loop (e.g., `locations[0][0] = 0.0`)
- **Observation:** Note the exact `TypeError` crash that Python triggers to protect immutable data

CRUD Operations & Summary

Retrieval, Updates, and Collection Iteration

- **Retrieve:** Safely check key presence before extracting quantities
- **Update:** Target nested value coordinates to mutate state
- **Iterate:** Use `.items()` to unpack parent and child fields together

Core Iteration Pattern:

Unpacking keys and nested containers cleanly

```
for item_name, details in inventory.items():
```

```
    print(f"{item_name}: {details['qty']}")
```

Lab Activity: Stock-Tracking Prototype

The Mission:

Building a Scalable Nested Inventory Structure

- **Goal:** Build a functional inventory dashboard engine
- **Core Collection:** Implement a nested dictionary pattern
- **Mapping Rule:** Map item string keys to inner property maps

Structural Strategy

Item Name Key -> Maps to inner attribute dictionaries

```
inventory = {  
    "Laptop": {  
        "price": 899.99,  
        "qty": 15  
    }  
}
```

Inventory Management Prototype



ACTIVITY



Objective: Build a small stock-tracking prototype

Tasks / Requirements:

- Store item data using a dictionary of dictionaries (item name -> {price, qty})
- Add functionality to retrieve and update a quantity
- Iterate through the inventory to print a summary

Key Point: Justify why a dictionary was chosen over a list for this scenario

Extension Activity: Add additional fields such as category or supplier



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation and Comparison Discussion

Analysing Data Structure Trade-Offs

The Three Evaluation Pillars

- **Flexibility vs. Structure:** Balancing the ability to mutate data easily against rigid structure enforcement
- **Read vs. Write Efficiency:** Analysing how choice of collection alters processor speed when looking up or saving data
- **Large-Scale Maintainability:** Evaluating how code adapts when data sets grow from three entries to three million

Expected Outcome

- **Explain Trade-Offs:** Articulate precisely why a chosen structure excels in one area while introducing constraints in another

Flexibility vs. Rigid Structure

Mutable Flexibility (Lists & Dictionaries):

- **The Good:** Dynamic scaling allows items to be added, removed, or updated on the fly
- **The Risk:** Free-form editing leaves records open to accidental overwrite bugs or data type inconsistencies

Structural Protection:

- **The Good:** Write-protected memory blocks guarantee data remains unchanged throughout program lifecycles
- **The Risk:** Absolute immutability requires copying and recreating objects if values ever need to change (**Tuples**)

Read vs. Write Efficiency

Sequential Scans (Lists):

- **Write:** Fast - items simply append straight to the end of array memory
- **Read:** Slow - finding a specific item forces a Linear Time check through every single slot

Direct Hashing Maps (Dictionaries):

- **Write:** Slight memory overhead needed to compute unique internal hash keys
- **Read:** Instant - fetching by direct key identifier scales at lightning-fast Constant Time

Maintainability in Larger Systems

**Schema
Rigidity**

**Team
Readability
Impact**

Stretch and Challenge

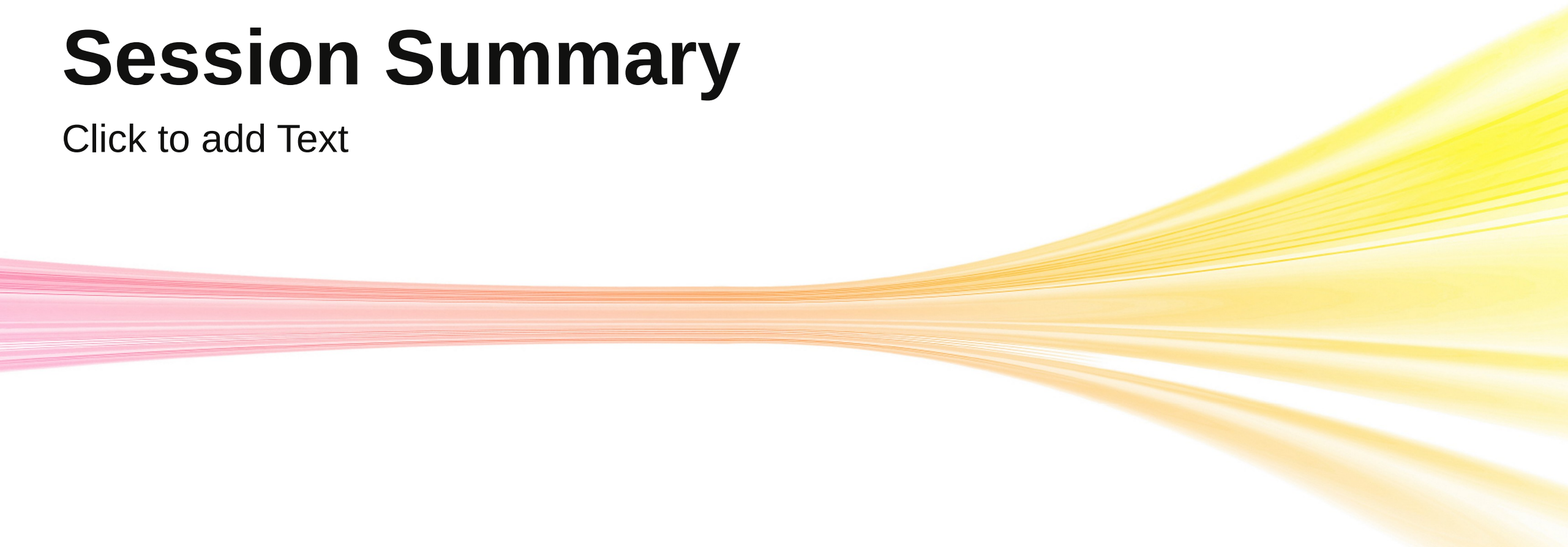
The Mission:

Upgrade your Inventory Management Prototype into production-grade code

- **Task 1:** Explore abstract structures such as stacks and queues
- **Task 2:** Map these concepts to Python implementations using lists
- **Task 3:** Explain how these patterns are used in real-world applications

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Data Structures and Their Application

Lesson Objectives

Apply

- Apply appropriate data structures to store and organise data

Analyse

- Analyse differences between lists, dictionaries, and tuples

Evaluate

- Evaluate structure suitability based on problem context

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, **K7**, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, S7, S10,
S11, S12, **S16**

Behaviours

B2, B3

